

Translated

From: e-maxx.ru

Fast Fourier transform

In this article we will discuss an algorithm that allows us to multiply two polynomials of length n in $O(n \log n)$ time, which is better than the trivial multiplication which takes $O(n^2)$ time. Obviously also multiplying two long numbers can be reduced to multiplying polynomials, so also two long numbers can be multiplied in $O(n \log n)$ time (where n is the number of digits in the numbers).

The discovery of the **Fast Fourier transformation (FFT)** is attributed to Cooley and Tukey, who published an algorithm in 1965. But in fact the FFT has been discovered repeatedly before, but the importance of it was not understood before the inventions of modern computers. Some researchers attribute the discovery of the FFT to Runge and König in 1924. But actually Gauss developed such a method already in 1805, but never published it.

Notice, that the FFT algorithm presented here runs in $O(n \log n)$ time, but it doesn't work for multiplying arbitrary big polynomials with arbitrary large coefficients or for multiplying arbitrary big integers. It can easily handle polynomials of size 10^5 with small coefficients, or multiplying two numbers of size 10^6 , which is usually enough for solving competitive programming problems. Beyond the scale of multiplying numbers with 10^6 bits, the range and precision of the floating point numbers used during the computation will not be enough to give accurate final results, though there are more complex variations that can perform arbitrary large polynomial/integer multiplications. E.g. in 1971 Schönhage and Strasser developed a variation for multiplying arbitrary large numbers that applies the FFT recursively in rings structures running in $O(n \log n \log \log n)$. And recently (in 2019) Harvey and van der Hoeven published an algorithm that runs in true $O(n \log n)$.

Discrete Fourier transform

Let there be a polynomial of degree $n - 1$:

$$A(x) = a_0 x^0 + a_1 x^1 + \dots + a_{n-1} x^{n-1}$$

Without loss of generality we assume that n - the number of coefficients - is a power of 2. If n is not a power of 2, then we simply add the missing terms $a_i x^i$ and set the coefficients a_i to 0.